Faculty of Computer Technology and Cybersecurity
Department of Computer Engineering
ARCHITECTURE AND ORGANIZATION OF COMPUTER SYSTEMS
WORKSHEET - 5

Full Name: Daniil Kalts

Group: IT2-2404SE

Q1: Discuss three categories of instruction set architecture in detail

Instruction set architectures can be grouped by where they keep operands for operations. The three common categories are stack, accumulator, and general-purpose register architectures.

- **Stack architecture:** Operands are kept on a stack. An operation such as ADD takes values from the top of the stack and puts the result back. Instructions can be short because they do not need to name every operand, but accessing a specific value lower in the stack is difficult.
- **Accumulator architecture:** One operand and the result are kept in a special register called the accumulator. For example, LOAD A puts A in the accumulator, and ADD B adds B to it. This is simple, but the accumulator can become a limitation because many operations depend on the same register.
- **General-purpose register architecture:** The CPU has several registers that can hold data and addresses. Instructions name the registers they use, so values can be kept close to the processor and reused. Some designs allow operations between registers and memory; load-store designs perform arithmetic only on registers and use separate instructions to load from or store to memory. DLX is a load-store design.

Q2: What are the instruction formats and what do we do in each instruction format?

An instruction format shows how the bits of an instruction are divided into fields. In DLX, the main formats are R-type, I-type, and J-type. The opcode identifies the operation, while other fields identify registers or an immediate value.

- **R-type (register):** Used for arithmetic and logical operations between registers, such as ADD or AND. Its fields include the opcode, source registers, destination register, and a function field that specifies the exact operation.
- **I-type (immediate):** Used when an instruction contains a constant value or an address offset. It is used for operations such as ADDI, load/store instructions, and conditional branches. Its fields include the opcode, register fields, and a 16-bit immediate value.
- **J-type (jump):** Used for jump instructions. It contains the opcode and a larger address field used to calculate the jump target. JAL is a J-type instruction.

The format tells the processor how to interpret the instruction bits and which registers, value, or address to use.

Q3: What is JAL instruction? Explain in detail

JAL means “Jump and Link.” It transfers program execution to a target address and saves the return address in register R31. A subroutine can use this saved address to return to the instruction after the JAL.

In DLX, the processor saves PC + 4 in R31, then calculates the jump target from the address following the JAL and its signed offset (the offset is shifted left by 2 because instructions are word-aligned). The program counter is set to this target, and execution continues there. When the subroutine is finished, it can jump to the address in R31.

For example, a program can use JAL to call a function. JAL remembers where the program should continue afterward, and the function returns through that saved address. R31 is the usual link register, so another call can overwrite it unless the program saves its value.

Q4: Explain five stages of execution

A DLX instruction passes through five stages. Each stage performs one part of the work:

- **IF – Instruction Fetch:** The processor uses the program counter (PC) to read the instruction from memory. It also prepares the address of the next instruction.
- **ID – Instruction Decode and Register Fetch:** The control unit interprets the instruction and reads the required source registers. It also prepares an immediate value when the instruction contains one.
- **EX – Execute or Address Calculation:** The ALU performs the required calculation. This can be arithmetic or logic, a comparison for a branch, or an effective memory address for a load/store.
- **MEM – Memory Access:** A load reads data from memory, and a store writes data to memory. Instructions that do not access data memory simply pass through this stage.
- **WB – Write Back:** The result is written to the destination register. For a load, the value from memory is written; for an arithmetic instruction, the ALU result is written.

Q5: What do we do in the datapath and how does it successfully implement instructions?

The datapath is the part of the processor that moves and processes data while instructions execute. It includes the program counter, instruction and data memories, register file, ALU, and connections between them. Control signals select the routes and operations.

For example, for an ADD instruction, the datapath fetches the instruction, reads two source registers, sends their values to the ALU, and writes the sum into the destination register. For a load instruction, it calculates an address, reads memory, and writes the loaded value into a register. The control unit decodes each instruction and tells the datapath which values to select, what operation to perform, and whether to read or write memory or a register. Together, the datapath and control unit implement the instruction set.

Q6: In the DLX processor every instruction goes through five clock cycles. Discuss in detail

In the basic five-stage DLX pipeline, one instruction needs five stages to complete: IF, ID, EX, MEM, and WB. If one instruction runs by itself without overlapping other instructions, these stages take five clock cycles, one stage per cycle.

With pipelining, different instructions can use different stages at the same time. For example, while one instruction is in EX, the next can be in ID and another can be in IF. After the pipeline fills, an ideal DLX processor can complete about one instruction per clock cycle, even though each individual instruction still takes about five cycles from fetch to write-back.

Not every instruction uses every stage for useful work. An arithmetic instruction does not need to read or write data memory, and a store does not write a result to a register. In the simple pipeline, these instructions still pass through the stages so later instructions can stay in order. Branches and hazards can also cause stalls, so real execution may take more cycles than the ideal case.